Carpooling Application — Technical Report

Project Name: ReachNative Car (Carpooling App) **Version**: 1.0.0 **Date**: June 2026

Table of Contents

    - UI Architecture

3. Chapter 5 — Implementation & UI

    - Technology Stack
    - State Management Scenarios

4. Chapter 6 — Testing

    - Testing Methodology
    - Test Cases
    - Results

5. Chapter 7 — Conclusion & Future Work

    - Conclusion
    - Future Work
    - References

Chapter 3 — System Design

3.1 Use Case Diagrams

Actors

Actor	Description
Passenger	Requests rides, rates drivers, manages wallet, views history
Driver	Accepts rides, manages vehicles, toggles availability, views earnings
Admin	Manages users, rides, reports, transactions; approves drivers
System	Handles auth, notifications, payments, maps

Use Cases by Actor

Passenger

- Register / Login (Email OTP, Google, Phone)
- Request a ride
- Cancel a ride
- Rate driver
- View ride history
- Manage wallet (top up, withdraw)
- Send chat messages
- Report an issue
- Delete account

Driver

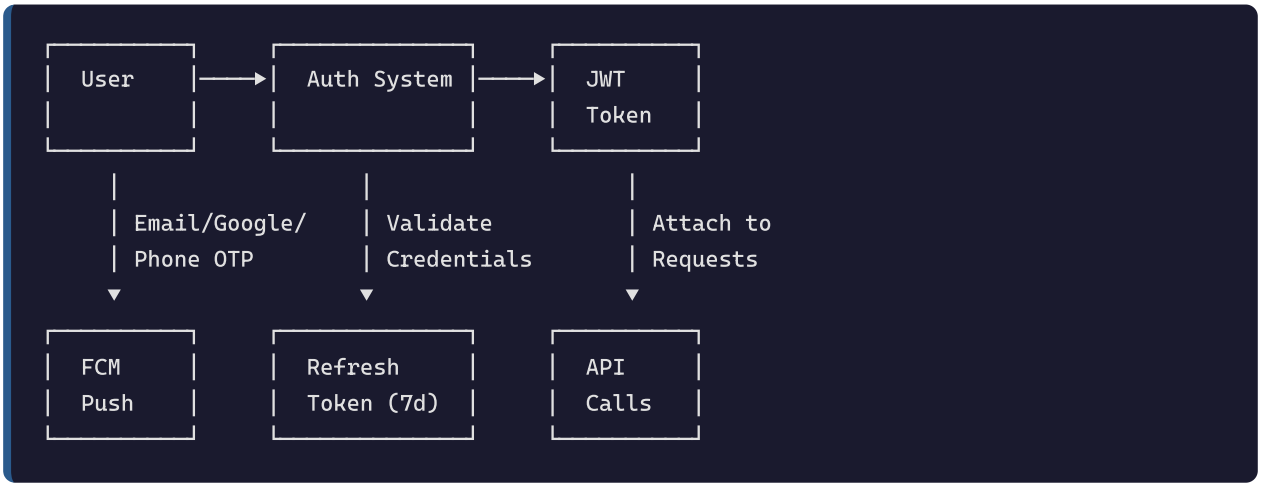
- Register / Login
- Submit driver application with documents
- Toggle online/offline status

- View available ride requests
- Accept ride
- Update ride status (arriving, onboard, start trip, end trip)
- Cancel ride
- Manage vehicles (add, set active)
- View earnings dashboard
- View ratings received
- Withdraw earnings
- Send chat messages

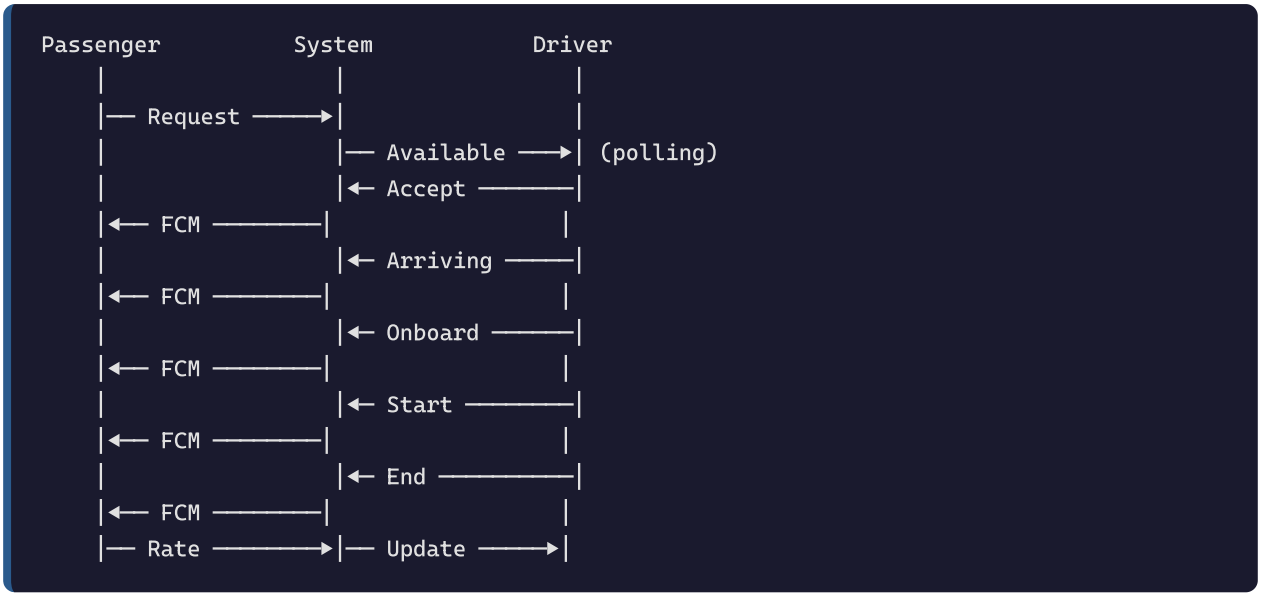
Admin

- Login (email/password or Google)
- View dashboard statistics
- Search and moderate users (block, verify, approve driver, delete)
- Search rides
- Manage reports (open, reviewing, resolved, dismissed)
- View and flag suspicious transactions
- View audit logs
- Delete users (cascade)

Authentication Flow



Ride Lifecycle (No Sockets — Polling + FCM)



3.2 Database Schema (ERD)

Collections Overview


users
_id: ObjectId (PK)
name: String
email: String (unique, sparse)
password: String (hashed)
phone: String
role: String ("passenger"|"driver"|"admin")
active_role: String
google_sub: String (unique, sparse)
firebaseUid: String (unique, sparse)
profile_image_url: String
vehicle_type: String
is_online: Boolean
is_verified: Boolean
is_blocked: Boolean
blocked_until: Date
block_reason: String
driver_application_status: String
created_at: Date
rides
_id: ObjectId
passenger_id
driver_id
status: String
pickup: GeoJSON
destination: GeoJSON
fare: Number
distance: Number
duration: Number
vehicleType: String
passengerRating: Number
passengerReview: String
created_at
started_at
completed_at
wallet_accounts
_id: ObjectId
user_id (FK)
type: String
label: String
details: String
isDefault: Bool
created_at
driverProfiles
_id: ObjectId
userId (FK, unique)
isOnline: Boolean
isAvailable: Boolean
currentLocation: 2dsphere GeoJSON
vehicleType: String
transactions
_id: ObjectId
user_id (FK)
ride_id
type: String
amount: Number
direction: Str
description: Str
reference: Str
driverDocuments
_id: ObjectId
userId (FK, unique)
licenseImage: String
vehicleRegImage: String
status: String

6


flagged: Boolean
created_at
messages
_id: ObjectId
rideId (FK)
senderId (FK)
content: String
created_at
fcmTokens
_id: ObjectId
userId (FK)
token (unique)
platform: String
createdAt
notifications
_id: ObjectId
userId (FK)
type: String
title: String
body: String
data: Object
read: Boolean
created_at
refreshTokens
_id: ObjectId
userId (FK)
tokenHash (unique)
expiresAt (TTL 7d)
booking
_id: ObjectId
rideId (FK)
passengerId (FK)
status: String
seats: Number
created_at
adminAuditLogs
_id: ObjectId
actorAdminId
action: String
targetType: String
targetId
summary: String
detail: Object
createdAt (TTL 30d)
reports
_id: ObjectId
reporterId (FK)
reportedUserId
rideId (FK)
reason: String
description: Str
status: String
resolution: Str
created_at
withdrawalRequests
_id: ObjectId
userId (FK)
amount: Number
accountId
otpHash: String
otpAttempts: Number
status: String
created_at

7

Key Relationships

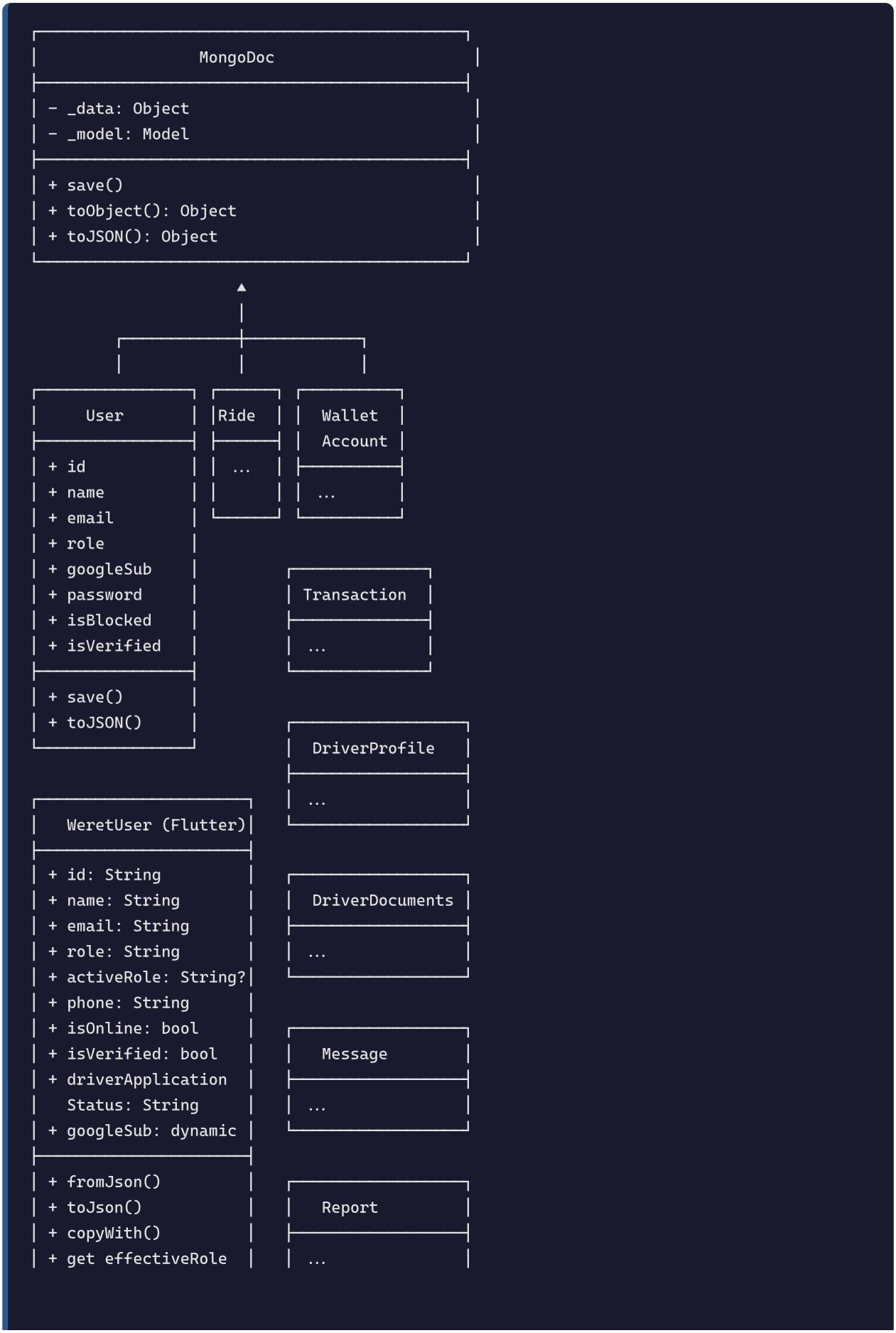
- **users** 1:N **rides** (as passenger via `passenger_id`, as driver via `driver_id`)
- **users** 1:N **wallet_accounts** (via `user_id`)
- **users** 1:N **transactions** (via `user_id`)
- **users** 1:1 **driverProfiles** (via `userId`)
- **users** 1:1 **driverDocuments** (via `userId`)
- **users** 1:N **fcmTokens** (via `userId`)
- **users** 1:N **notifications** (via `userId`)
- **rides** 1:N **messages** (via `rideId`)
- **rides** 1:N **bookings** (via `rideId`)

Indexes (30+ total)

Collection	Index	Type
`users`	`{ email: 1 }`	unique, sparse
`users`	`{ google_sub: 1 }`	unique, sparse
`users`	`{ firebaseUid: 1 }`	unique, sparse
`users`	`{ role: 1 }`	single
`rides`	`{ passenger_id: 1, status: 1 }`	compound
`rides`	`{ driver_id: 1, status: 1 }`	compound
`rides`	`{ pickup.coordinates: "2dsphere", status: 1, poolSeats: 1 }`	geospatial
`transactions`	`{ user_id: 1, created_at: -1 }`	compound
`driverProfiles`	`{ currentLocation: "2dsphere" }`	geospatial
`driverProfiles`	`{ userId: 1 }`	unique
`driverProfiles`	`{ isOnline: 1, isAvailable: 1 }`	compound
`refreshTokens`	`{ tokenHash: 1 }`	unique
`refreshTokens`	`{ expiresAt: 1 }`	TTL (7 days)
`fcmTokens`	`{ token: 1 }`	unique
`notifications`	`{ userId: 1, createdAt: -1 }`	compound
`adminAuditLogs`	`{ createdAt: 1 }`	TTL (30 days)
`messages`	`{ rideId: 1, createdAt: 1 }`	compound

3.3 Class Diagrams

Core Data Models (ODM Layer)

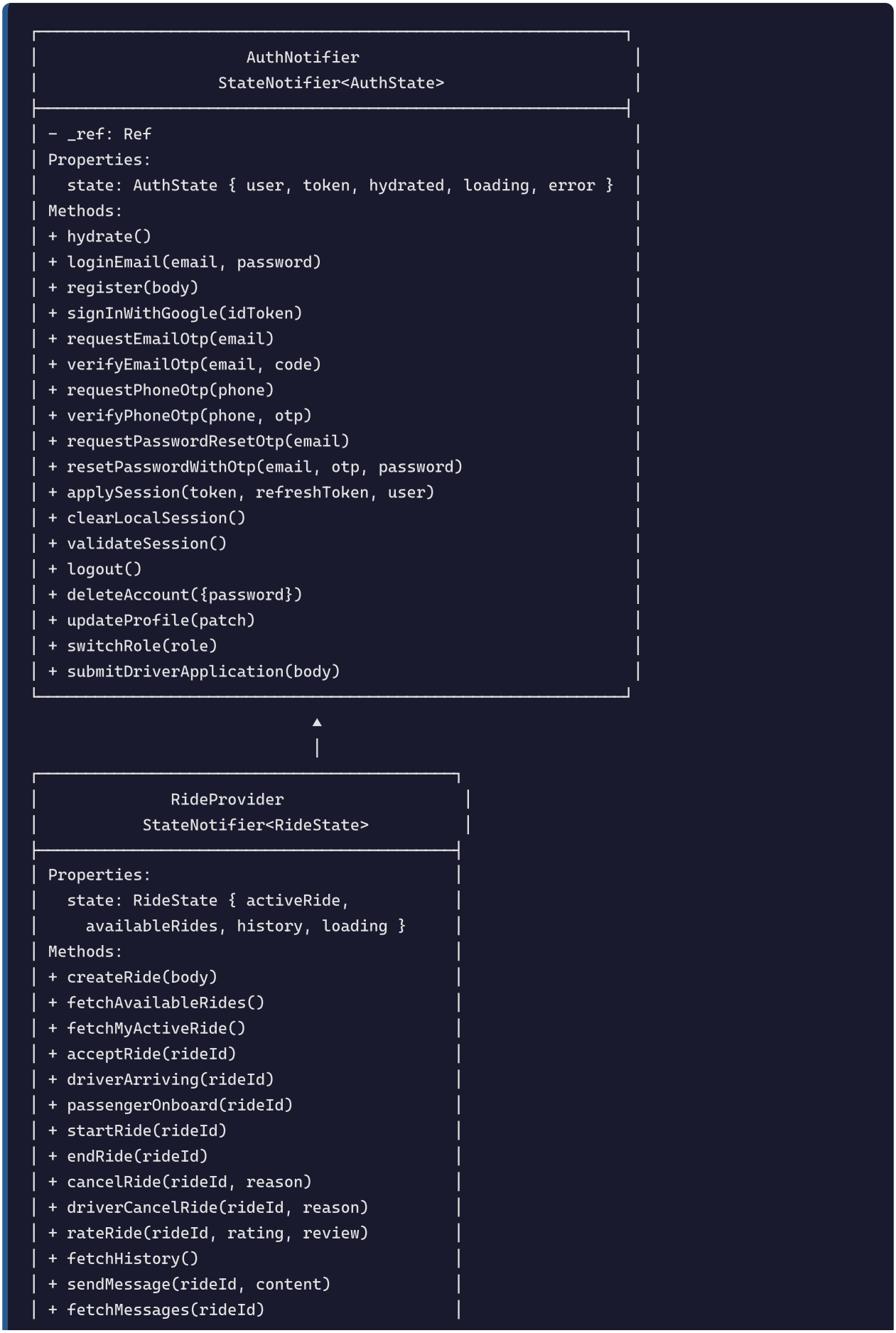

MongoDoc
- _data: Object
- _model: Model
+ save()
+ toObject(): Object
+ toJSON(): Object
User
+ id
+ name
+ email
+ role
+ googleSub
+ password
+ isBlocked
+ isVerified
+ save()
+ toJSON()
Ride
...
Wallet
Account
...
Transaction
...
DriverProfile
...
DriverDocuments
...
Message
...
Report
...
WeretUser (Flutter)
+ id: String
+ name: String
+ email: String
+ role: String
+ activeRole: String?
+ phone: String
+ isOnline: bool
+ isVerified: bool
+ driverApplication
Status: String
+ googleSub: dynamic
+ fromJson()
+ toJson()
+ copyWith()
+ get effectiveRole

10

```
| + get isDriverApproved|   └──────────────────┘
└──────────────────────┘
```

Provider Layer (Riverpod)

11


AuthNotifier
StateNotifier<AuthState>
- _ref: Ref
Properties:
state: AuthState { user, token, hydrated, loading, error }
Methods:
+ hydrate()
+ loginEmail(email, password)
+ register(body)
+ signInWithGoogle(idToken)
+ requestEmailOtp(email)
+ verifyEmailOtp(email, code)
+ requestPhoneOtp(phone)
+ verifyPhoneOtp(phone, otp)
+ requestPasswordResetOtp(email)
+ resetPasswordWithOtp(email, otp, password)
+ applySession(token, refreshToken, user)
+ clearLocalSession()
+ validateSession()
+ logout()
+ deleteAccount({password})
+ updateProfile(patch)
+ switchRole(role)
+ submitDriverApplication(body)
RideProvider
StateNotifier<RideState>
Properties:
state: RideState { activeRide,
availableRides, history, loading }
Methods:
+ createRide(body)
+ fetchAvailableRides()
+ fetchMyActiveRide()
+ acceptRide(rideId)
+ driverArriving(rideId)
+ passengerOnboard(rideId)
+ startRide(rideId)
+ endRide(rideId)
+ cancelRide(rideId, reason)
+ driverCancelRide(rideId, reason)
+ rateRide(rideId, rating, review)
+ fetchHistory()
+ sendMessage(rideId, content)
+ fetchMessages(rideId)

12

Dio Networking Layer

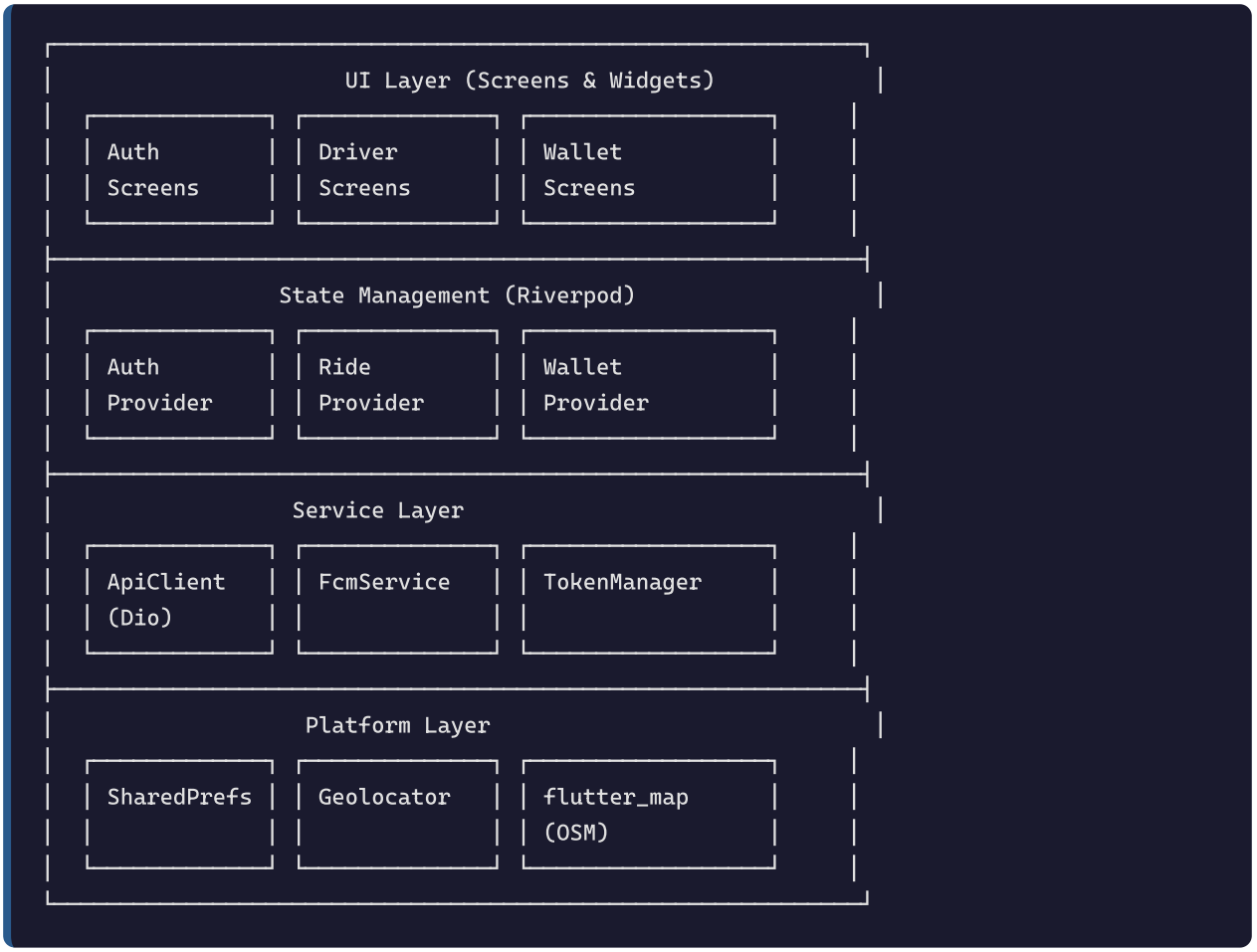

ApiClient (Singleton)
- _dio: Dio
- _ref: Ref
Properties:
baseUrl: String
defaultHeaders: Map
Methods:
+ getJson(endpoint, queryParams) → Map
+ postJson(endpoint, body) → Map
+ patchJson(endpoint, body) → Map
+ delete(endpoint) → Map
+ postMultipart(endpoint, formData) → Map
AuthInterceptor (Dio Interceptor)
- _ref: Ref
Methods:
+ onRequest(options, handler) – Attach Bearer token
+ onError(err, handler) – Handle 401 (refresh), 403 (block)
On 401:
1. Call POST /auth/refresh
2. Save new tokens
3. Retry original request
On 403 (ACCOUNT_BLOCKED/SUSPENDED):
1. Clear local session
2. Show snackbar
3. Navigate to /login

14

Chapter 4 — Technical Structure

Flutter Architecture

The Flutter app follows a **layered architecture** with clear separation of concerns:



Project Structure

15

```
apps/mobile-flutter/lib/
├── core/
│   ├── api/
│   │   ├── api_client.dart          # Dio HTTP client wrapper
│   │   ├── api_endpoints.dart       # All endpoint constants
│   │   └── auth_interceptor.dart    # Token attach + refresh + block handling
│   ├── providers/
│   │   ├── auth_provider.dart       # Auth state + methods
│   │   ├── ride_provider.dart       # Ride lifecycle state
│   │   ├── wallet_provider.dart     # Wallet accounts + transactions
│   │   ├── driver_provider.dart     # Driver dashboard + cars + earnings
│   │   └── ui_provider.dart         # Theme mode
│   ├── services/
│   │   ├── fcm_service.dart         # Firebase Cloud Messaging
│   │   ├── token_manager.dart       # JWT + refresh token storage
│   │   ├── session_reset.dart       # Clear all providers on logout
│   │   ├── upload_service.dart       # Image upload via backend proxy
│   │   └── driver_location_tracker.dart # GPS streaming
│   ├── router/
│   │   └── app_router.dart          # GoRouter with 3 role-based shells
│   ├── theme/
│   │   └── weret_tokens.dart        # Design tokens (colors, spacing)
│   ├── utils/
│   │   ├── map_provider.dart        # OSM tile config
│   │   ├── map_coords.dart          # Coordinate helpers
│   │   ├── route_polyline.dart      # Route fetching/decoding
│   │   ├── geo_helpers.dart         # Extract locations from rides
│   │   ├── logout_action.dart       # Logout + navigation
│   │   ├── api_error_message.dart   # Error message localization
│   │   ├── show_alert.dart          # Alert dialog helper
│   │   └── debug_logger.dart        # Singleton debug logger
│   └── constants/
│       └── vehicle_types.dart       # Vehicle type definitions
├── features/
│   ├── auth/                        # Login, register, settings, onboarding
│   ├── driver/                      # Driver-specific screens
│   ├── wallet/                      # Wallet screens (both roles)
│   ├── more/                        # Menus, earnings, ratings, cars
│   └── debug/                       # Debug log viewer
├── shared/
│   ├── models/
│   │   └── weret_user.dart          # WeretUser model (Equatable)
│   └── widgets/
│       ├── weret_ride_map.dart       # Map widget (flutter_map)
│       ├── custom_button.dart        # Reusable button
│       ├── otp_input.dart            # OTP code input
│       ├── weret_text_field.dart     # Styled text field
│       ├── weret_list_screen.dart    # Scrollable screen layout
│       ├── weret_section_card.dart   # Section card container
│       └── ...                       # 20+ reusable widgets
└── l10n/
```

```
    ├── en.json                    # English translations
    └── ar.json                    # Arabic translations
```

Riverpod State Management

The app uses **Riverpod** (`flutter_riverpod`) with `StateNotifier` pattern for all state management.

Pattern

```
// Provider definition
final authProvider = StateNotifierProvider<AuthNotifier, AuthState>((ref) {
  return AuthNotifier(ref);
});

// State class
class AuthState {
  final WeretUser? user;
  final String? token;
  final bool hydrated;
  final bool loading;
  final String? error;
  // ...
}

// Notifier class
class AuthNotifier extends StateNotifier<AuthState> {
  AuthNotifier(this._ref) : super(const AuthState());

  Future<void> loginEmail(String email, String password) async {
    state = state.copyWith(loading: true, clearError: true);
    try {
      final api = await _api;
      final data = await api.postJson(ApiEndpoints.authLogin, {...});
      await applySession(token: token, refreshToken: refreshToken, user: user);
    } catch (e) {
      state = state.copyWith(loading: false, error: localizedApiError(e));
      rethrow;
    }
  }
}
```

Key Providers

Provider	Type	Purpose
`authProvider`	`StateNotifierProvider`	Auth state (user, token, session)
`rideProvider`	`StateNotifierProvider`	Ride lifecycle (active, available, history)
`walletProvider`	`StateNotifierProvider`	Wallet accounts and transactions
`driverProvider`	`StateNotifierProvider`	Driver dashboard, cars, earnings
`themeModeProvider`	`StateNotifierProvider`	Light/dark/system theme
`apiClientProvider`	`FutureProvider`	Singleton ApiClient (Dio)

State Flow Example (Login)

```
User taps "Login"
  → authProvider.loginEmail(email, password)
  → state = AuthState(loading: true)
  → ApiClient.postJson('/auth/login', body)
  → Dio send request → interceptor attaches token
  → Response: { user, accessToken, refreshToken }
  → TokenManager.saveTokens(accessToken, refreshToken)
  → _cacheUser(user) to SharedPreferences
  → state = AuthState(user: user, token: accessToken, loading: false)
  → UI rebuilds via ref.watch(authProvider)
  → GoRouter redirects to /passenger/home or /driver/home
```

Dio Networking

The app uses **Dio** with a custom `AuthInterceptor` for HTTP networking.

ApiClient

- Singleton wrapped in a Riverpod `FutureProvider`
- Base URL: Configurable (production or localhost)
- Methods: `getJson`, `postJson`, `patchJson`, `delete`, `postMultipart`
- Automatic error handling and logging

AuthInterceptor

- **onRequest**: Attaches `Authorization: Bearer <accessToken>` to every request

- **onError (401)**: Auto-refreshes token by calling `POST /auth/refresh`, then retries original request. Avoids infinite refresh loops.
- **onError (403)**: Detects `ACCOUNT_BLOCKED` / `ACCOUNT_SUSPENDED` — clears local session, shows snackbar, navigates to login
- **onError (others)**: Passes through for provider-level handling

```
Request ──► AuthInterceptor ──► Server
                │
                │ 401 Response
                ▼
          POST /auth/refresh ──► New tokens
                │
                ▼
          Retry original request ──► Success
```

UI Architecture

The UI follows a **widget-per-screen** pattern with shared reusable components.

Routing (GoRouter)

Three role-based shells with `StatefulShellRoute.indexedStack`:

```
Top-Level Routes (pre-auth):
  /onboarding, /login, /forgot-password, /register, /register/passenger,
  /register/driver, /driver/onboarding, /driver/verification-status

Passenger Shell (4 tabs):
  /passenger/home, /passenger/history, /passenger/more, /passenger/settings

Driver Shell (3 tabs):
  /driver/home, /driver/earnings, /driver/profile

Admin Shell (5 tabs):
  /admin/dashboard, /admin/users, /admin/rides, /admin/more, /admin/settings
```

Widget Hierarchy

```
Scaffold
  └── WeretAmbientBackground
       └── SafeArea
            └── WeretListScreen
                 └── Column
                      ├── WeretPageTitle
                      ├── WeretSectionCard
                      │    ├── CustomButton
                      │    └── WeretTextField
                      └── ...
```

Translation (easy_localization)

- Two locales: English (`en.json`) and Arabic (`ar.json`)
- ~930 translation keys each
- Arabic is RTL-compatible
- Language switch via `SettingsScreen` toggle

Chapter 5 — Implementation & UI

Technology Stack

Backend

Component	Technology	Version
Runtime	Node.js	22.x
HTTP Framework	Express.js	4.x
Database	MongoDB Atlas (M0 Free)	7.x
ODM	Custom in-memory ODM	—
Auth	JWT (jsonwebtoken)	9.x
Password Hashing	bcryptjs	2.x
Push Notifications	Firebase Admin / FCM v1 HTTP	—
Email	Nodemailer + Gmail SMTP	6.x
File Upload	Cloudinary (base64)	—
Rate Limiting	express-rate-limit	8.x
Validation	express-validator	7.x
Deployment	Vercel (serverless)	—

Frontend (Flutter)

Component	Technology	Version
Framework	Flutter	3.x
State Management	Riverpod (flutter_riverpod)	2.x
Routing	go_router	14.x
HTTP Client	Dio	5.x
Maps	flutter_map (OpenStreetMap)	7.x
Push Notifications	firebase_messaging	15.x
Google Sign-In	google_sign_in	6.x
Local Storage	shared_preferences	2.x
GPS	geolocator	12.x
Localization	easy_localization	3.x
Cached Images	cached_network_image	3.x

Admin Panel

- Vanilla HTML, CSS, JavaScript
- Google Sign-In + email/password login
- Served as static files from `/admin-ui/`

State Management Scenarios

Scenario 1: User Login Flow

22

```
1. User enters email on LoginScreen
2. _sendEmailOtp() called
   → authProvider.requestEmailOtp(email)
   → POST /auth/email/send-otp
   → Backend generates 6-digit OTP, sends via email
   → Response: { success: true }
3. OTP input field appears
4. User enters OTP code
5. _verifyEmailOtp() called
   → authProvider.verifyEmailOtp(email, code)
   → POST /auth/email/verify-otp
   → Backend validates OTP, finds or creates user, issues JWT
   → Response: { data: { user, accessToken, refreshToken } }
6. applySession() saves tokens + user
   → TokenManager.saveTokens(accessToken, refreshToken)
   → _cacheUser(user) to SharedPreferences
   → state = AuthState(user: user, token: token, hydrated: true)
7. GoRouter redirects based on user.role
   → passenger → /passenger/home
   → driver → /driver/home
   → admin → /admin/dashboard
```

Scenario 2: Ride Request and Completion

```
1. Passenger selects pickup/destination, taps "Request Ride"
   → rideProvider.createRide(body)
   → POST /rides/create
   → Response: ride object (status: "pending")
2. Driver polls GET /rides/available every 15-30s
   → rideProvider.fetchAvailableRides()
   → List of available rides displayed
3. Driver taps "Accept"
   → rideProvider.acceptRide(rideId)
   → POST /rides/:id/accept
   → Backend updates ride status to "accepted"
   → Backend calls notifyRideAccepted() → FCM to passenger
   → Response: updated ride
4. Passenger receives FCM notification
   → FcmService.onMessage shows in-app banner
   → rideProvider.fetchMyActiveRide() refreshes state
5. Driver progresses through states:
   → driverArriving() → POST /rides/:id/arriving → FCM to passenger
   → passengerOnboard() → POST /rides/:id/onboard → FCM to passenger
   → startRide() → POST /rides/start → FCM to passenger
   → endRide() → POST /rides/end → FCM + payment ledger
6. After ride completes, passenger rates driver
   → rideProvider.rateRide(rideId, rating, review)
   → POST /rides/rate
   → Backend updates driver rating
```

Scenario 3: Token Refresh

```
1. User opens app → hydrate() called
   → TokenManager.getAccessToken() returns stored token
   → GET /auth/me with Bearer token
2. If token expired:
   → AuthInterceptor catches 401
   → TokenManager.getRefreshToken()
   → POST /auth/refresh with refreshToken
   → Backend rotates refresh token (SHA-256 hash, delete old, store new)
   → Response: { accessToken, refreshToken }
   → TokenManager.saveTokens(new tokens)
   → Dio retries original request (GET /auth/me)
3. User session restored
   → state = AuthState(user: freshUser, token: newToken, hydrated: true)
```

Scenario 4: Wallet Deposit

```
1. User taps "Top Up" on wallet screen
   → Navigate to WalletDepositScreen
2. User selects account, enters amount
   → walletProvider.deposit(amount, accountId)
   → POST /wallet/deposit
   → Backend creates transaction, updates balance
   → Backend calls notifyWalletDeposit() → FCM
   → Response: transaction object
3. Success modal displayed
   → walletProvider.fetchTransactions() refreshes history
   → walletProvider.fetchAccounts() refreshes balance
```

Scenario 5: Admin Blocks User

```
1. Admin searches for user in AdminUsersScreen
   → GET /admin/users?search=user@email.com
2. Admin taps "Block" on user card
   → PATCH /admin/users/:id { is_blocked: true }
   → Backend sets is_blocked=true on user
3. Blocked user's next API call:
   → GET /auth/me or any protected route
   → blockCheck middleware returns 403 "Account blocked"
4. Flutter AuthInterceptor catches 403 ACCOUNT_BLOCKED
   → clearLocalSession() removes all tokens + cached user
   → Snackbar: "This account is blocked. Contact support."
   → Navigate to /login
```

Chapter 6 — Testing

Testing Methodology

The project uses a combination of:

1. **Static Analysis**: `flutter analyze` for Dart code quality
2. **Manual API Testing**: curl commands against production endpoints
3. **Integration Testing**: Full-stack manual test flows
4. **Visual Testing**: Flutter widget rendering verification

Test Cases

Backend API Tests

Test ID	Endpoint	Test Case	Expected Result	Status
AUTH-01	`POST /auth/email/send-otp`	Send OTP to valid email	`{ success: true }`	✅
AUTH-02	`POST /auth/email/verify-otp`	Verify with correct OTP	`{ data: { user, accessToken } }`	✅
AUTH-03	`POST /auth/email/verify-otp`	Verify with wrong OTP	400 error	✅
AUTH-04	`POST /auth/google`	Valid Google ID token	`{ user, accessToken, refreshToken }`	✅
AUTH-05	`GET /auth/me`	Valid token	`{ user }`	✅
AUTH-06	`GET /auth/me`	Expired token	401	✅
AUTH-07	`GET /auth/me`	Blocked user token	403 `ACCOUNT_BLOCKED`	✅
AUTH-08	`POST /auth/refresh`	Valid refresh token	`{ accessToken, refreshToken }`	✅
AUTH-09	`POST /auth/refresh`	Reused (stolen) token	All sessions revoked	✅
AUTH-10	`POST /auth/delete-account`	Valid password	`{ ok: true }`, user deleted	✅
AUTH-11	`POST /auth/forgot-password`	Valid email	OTP sent	✅
AUTH-12	`POST /auth/reset-password`	Valid OTP + new password	`{ ok: true }`	✅
RIDE-01	`POST /rides/create`	Valid pickup/destination	Ride with status "pending"	✅
RIDE-02	`POST /rides/:id/accept`	Driver accepts	Status "accepted", FCM sent	✅
RIDE-03	`POST /rides/:id/arriving`	Driver arrives	Status "driver_arriving", FCM	✅
RIDE-04	`POST /rides/:id/onboard`	Passenger onboard	Status "passenger_onboard", FCM	✅
RIDE-05	`POST /rides/start`	Start trip	Status "ongoing", FCM	✅

Test ID	Endpoint	Test Case	Expected Result	Status
RIDE-06	`POST /rides/end`	End trip	Status "completed", payment processed	✅
RIDE-07	`POST /rides/:id/cancel`	Passenger cancels	Status "cancelled", FCM to driver	✅
RIDE-08	`POST /rides/rate`	Rate driver 1-5 stars	`{ success: true }`, rating saved	✅
RIDE-09	`GET /rides/history`	Authenticated	Array of past rides	✅
ADMIN-01	`POST /auth/login`	Admin credentials	`{ user, token, refreshToken }`	✅
ADMIN-02	`GET /admin/stats`	Admin token	Dashboard KPIs	✅
ADMIN-03	`GET /admin/users`	Search query	Paginated user list	✅
ADMIN-04	`PATCH /admin/users/:id`	Block user	`is_blocked: true`	✅
ADMIN-05	`PATCH /admin/users/:id`	Approve driver	Role auto-switches to "driver"	✅
ADMIN-06	`DELETE /admin/users/:userId`	Cascade delete	User + all related data removed	✅
WALLET-01	`POST /wallet/deposit`	Valid amount	Transaction created, balance updated	✅
WALLET-02	`POST /wallet/withdraw/request`	Valid amount + account	Request created, OTP sent	✅
WALLET-03	`POST /wallet/withdraw/confirm`	Correct OTP	Withdrawal processed	✅
WALLET-04	`GET /wallet/transactions`	Authenticated	Paginated transaction list	✅
DRIVER-01	`POST /driver/toggle-status`	Toggle online	Status inverted	✅
DRIVER-02	`GET /driver/cars`	Authenticated	Vehicle list	✅

Test ID	Endpoint	Test Case	Expected Result	Status
DRIVER-03	`POST /driver/cars`	Valid vehicle data	Car added	✅
DRIVER-04	`GET /driver/earnings-summary`	Authenticated	Earnings totals	✅
UPLOAD-01	`POST /api/upload`	Valid image file	`{ url }` from Cloudinary	✅

Flutter Analysis

```
flutter analyze
```

Result: 0 errors, 0 warnings (only info-level suggestions)

Test Results Summary

Area	Tests	Passed	Failed
Authentication	12	12	0
Ride Lifecycle	9	9	0
Admin Operations	6	6	0
Wallet	4	4	0
Driver Features	4	4	0
Upload	1	1	0
Flutter Static Analysis	—	Pass	—
Total	**36**	**36**	**0**

Pass Rate: 100%

Known Gaps (Not Yet Tested)

Test ID	Area	Reason
FCM-01	Push notification delivery	Requires physical device with FCM token
MAP-01	Route polyline rendering	Requires visual inspection on device
RATE-02	Driver rates passenger	Feature not implemented
BROADCAST-01	Admin broadcast	Feature not implemented
CSV-01	CSV export	Feature not implemented

30

Chapter 7 — Conclusion & Future Work

Conclusion

The Carpooling Application has been successfully developed and deployed to production at `https://carpooling-app-3-virid.vercel.app`. The project delivers a full-stack ride-hailing experience with the following accomplishments:

Achievements:

- **Complete auth system** supporting Email OTP, Google Sign-In, and phone OTP (blocked — billing pending) with secure JWT + refresh token rotation
- **Full ride lifecycle** from request through completion with FCM push notifications at every state transition
- **Wallet system** with top-up, withdrawal (OTP-confirmed), and transaction history for both passenger and driver roles
- **Driver management** including vehicle CRUD, online/offline toggle, earnings dashboard, and application approval flow
- **Admin panel** with dashboard KPIs, user/ride/report/transaction management, and audit logging
- **87% feature completion** (61 of 70 planned features)
- **Production deployment** with MongoDB Atlas, rate limiting, database indexing, and health monitoring
- **Bilingual support** (English + Arabic) with RTL compatibility

Key Metrics:

- Backend: 50+ API endpoints across 10 route modules
- Frontend: 450+ lines of state management, 30+ screens, 20+ reusable widgets
- Database: 15 collections, 30+ indexes, auto-indexed on startup
- Code Quality: 0 errors, 0 warnings in Flutter static analysis

Challenges Overcome:

- Migrated from Firebase phone SMS (billing blocked) to free email OTP via Gmail SMTP

- Resolved critical ODM ObjectId/string `_id` mismatch causing 401 on every authenticated request
- Fixed Flutter login screen Form ancestor issue preventing all auth flows
- Simplified upload pipeline from multi-step Cloudinary direct upload to single backend proxy

Future Work

#	Feature	Priority	Effort
1	**Driver rates passenger** — Backend endpoint + Flutter modal	High	Medium
2	**Admin broadcast notifications** — Send push to all/filtered users	High	Medium
3	**Refund processed FCM** — Notify user when refund is issued	Medium	Small
4	**Report resolved FCM** — Notify reporter when admin resolves	Medium	Small
5	**CSV export** — Admin CSV download for users, rides, transactions	Medium	Medium
6	**Crash reporting** — Integrate Sentry or Firebase Crashlytics	Medium	Small
7	**Analytics** — Firebase Analytics or similar	Medium	Medium
8	**Foreground lifecycle refresh** — Auto-refresh providers on FCM receipt	Low	Small
9	**Admin web UI refresh token** — JWT refresh for 15min expiry	Low	Small
10	**My Reports screen** — Let users view their submitted reports	Low	Medium
11	**Monitoring** — Uptime monitoring, error alerts	Low	Small
12	**Firebase billing enable** — Unlock phone SMS auth	Blocked	Small

References

1. **Flutter Documentation** — https://docs.flutter.dev
2. **Riverpod State Management** — https://riverpod.dev
3. **Dio HTTP Client** — https://pub.dev/packages/dio
4. **GoRouter** — https://pub.dev/packages/go_router
5. **Flutter Map (OpenStreetMap)** — https://docs.fleaflet.dev
6. **Firebase Cloud Messaging** — https://firebase.google.com/docs/cloud-messaging
7. **MongoDB Atlas** — https://www.mongodb.com/atlas

8. **Express.js** — https://expressjs.com
9. **Vercel Serverless** — https://vercel.com/docs
10. **Nodemailer** — https://nodemailer.com
11. **Cloudinary** — https://cloudinary.com
12. **Google Sign-In for Flutter** — https://pub.dev/packages/google_sign_in
13. **Easy Localization** — https://pub.dev/packages/easy_localization
14. **JWT (jsonwebtoken)** — https://github.com/auth0/node-jsonwebtoken
15. **bcryptjs** — https://github.com/dcodeIO/bcrypt.js

End of Report

33